PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview**Error! Bookmark not defined.**2. Background & Business Objective 43. Product Purpose 54. System Functionalities 55. User Stories & Use Cases 76. Features Included (Scope Definition) 107. Features Not Included (Scope Control) **Error! Bookmark not defined.**8. Assumptions & Constraints 119. Risks & Questions Throughout Development 11

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: OracleSort— *AI-Powered Exam Workflow Orchestrator*

Version: 1.0

1. Project Overview

- **Problem Statement:** Secondary students (SPM/IGCSE/STPM) and tertiary students need to study past year questions organized by chapter — this method is proven highly effective for exam success. According to statistics from SPM top achievers interview, the majority of high-achieving students agreed that past year questions organized by chapter are very effective for revision and securing good results. However, manually sorting 10+ past year papers across 8-12 chapters takes 10-15 hours per subject — most students abandon this proven method due to time friction. Teachers who want to prepare chapter-organized question banks for their students lack automated tools, forcing them to spend weekends manually cutting and pasting PDFs. The result: a proven effective revision strategy is underutilized because the workflow is too painful.
- **Target Domain:** AI Systems & Agentic Workflow Automation — transforming manual exam preparation into an intelligent, multi-step orchestrated workflow where GLM serves as the central reasoning engine.

- **Proposed Solution Summary:** OracleSort is an AI-powered exam workflow engine serving three user roles. For teachers, upload past papers and GLM sorts every question by chapter — export chapter-sorted question banks as PDFs and distribute to students in seconds instead of weekends. For secondary students (SPM/IGCSE/STPM), upload past papers and get instant chapter-by-chapter organization, trend analysis, and predicted focus areas — revise efficiently using the proven "questions by chapter" method. For tertiary students, the same workflow applies to university exam patterns. What SPM data proves is effective — OracleSort makes it effortless.

© UMHackathon 2026
Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** Manual exam preparation follows a broken, time-intensive pattern. A student collects 5-10 past year papers, manually reads each question, guesses which chapter it belongs to, and creates chapter-wise notebooks — taking 10-15 hours per subject. Most students quit halfway. SPM revealation data confirms that students who use chapter-organized past questions achieve better results, but most don't do it because it takes too much time. Teachers face a parallel problem — they want to provide chapter-sorted question banks to students but must manually cut and paste from 5-10 PDFs per subject. For 5 subjects across 10 papers and 8 chapters, that's 400 manual operations per exam season. Most teachers give up and don't provide chapter banks at all. Tertiary students face even more complexity with longer papers and less structured support.
- **Importance of Solving This Issue:** For teachers, OracleSort reduces weekend work from 4-6 hours to 2 minutes per subject, ensures consistent quality with no missing questions, and scales across all subjects they teach. For secondary students, it lets them use the proven "questions by chapter" method without the time cost, eliminates manual chapter guessing with GLM-powered matching, and reduces exam anxiety with data-driven confidence. For tertiary students, it provides the same friction-free workflow for university-level exam patterns. For the Malaysian education system, it creates equity where every student gets the same quality of chapter-organized analysis regardless of school or tuition access, while reducing teacher burnout from manual PDF manipulation.
- **Strategic Fit / Impact:** OracleSort aligns with Domain 1 requirements — GLM understands unstructured PDF inputs, performs multi-step reasoning from extraction to prediction, orchestrates dynamic task decisions, generates structured outputs like chapter-sorted PDFs, maintains state across sessions, and is entirely GLM-centric (remove GLM = system useless). The engine serves three user roles: teachers upload papers and export chapter banks for students, secondary students upload papers and get organized questions with predictions, and tertiary students follow the same workflow. Token economics show average session at ~15-30K tokens, with 50M token grant supporting ~1,500-3,000

© UMHackathon 2026

sessions-- cost is managed by calling GLM only for chapter detection, question matching, and prediction generation.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-powered workflow engine that automates the organization of past year questions by chapter, enabling teachers to easily prepare chapter-sorted question banks and students to conduct effective, data-driven revision using the proven "questions by chapter" method.

3.2. Intended Users (Target Audience):

- **Secondary Students (SPM/IGCSE/STPM):** Students preparing for major examinations who want to use the proven method of studying past year questions by chapter but lack the time to manually organize papers themselves.
- **Tertiary Students (University/College):** University students facing complex, multi-chapter subjects who need efficient revision tools when lecturers don't provide organized question banks.
- **Teachers (Secondary & Tertiary):** Educators who want to prepare high-quality chapter-organized question banks for their students but currently spend weekends manually cutting and pasting PDFs.

4. System Functionalities

4.1. Description: OracleSort operates as an AI-powered workflow engine that transforms unstructured exam papers into organized, chapter-sorted question banks. Teachers and students upload PDFs of past year papers and lecture slides. The GLM model extracts text, detects chapter boundaries from lecture materials, matches each question to the correct chapter, analyzes frequency trends across years, generates exam predictions with reasoning, and exports chapter-sorted PDFs. The entire multi-step workflow is orchestrated by GLM as the central reasoning engine.

© UMHackathon 2026

4.2. Key Functionalities:

- **Subject Folders** — Create and manage subjects. Data saves permanently.
- **Batch Upload & One-Click Analysis** — Upload multiple past papers and lecture slides at once. Press one button. GLM detects chapters, matches every question, and generates predictions.
- **Trend Dashboard & Prediction Cards** — Bar charts show question frequency per chapter. High/Medium/Low priority predictions with AI reasoning.
- **Reference Panel** — Click any result. Shows source document, original question, and year.
- **Export (Separate or Combined)** — Download PDFs sorted by chapter. Individual files per chapter or one merged PDF in your chosen order.
- **AI Chat Assistant** — Ask questions about your analysis and study strategies. Get instant answers.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: OracleSort uses Z AI's GLM-5.1 via https://api.ilmu.ai/v1/chat/completions .This model was selected because it provides strong reasoning capabilities for multi-step workflows, handles unstructured PDF text extraction reliably, supports JSON output formatting for structured data, and offers sufficient capacity for the hackathon. GLM-5.1's ability to perform chain-of-thought reasoning is essential for matching ambiguous questions to correct chapters.

4.3.2. **Prompting Strategy:** OracleSort uses multi-step agentic prompting. Instead of a single prompt, the system breaks the workflow into sequential GLM calls — first extract chapter boundaries from lecture slides, then classify each question against those chapters, then analyze frequency trends, then generate predictions. Each step passes context from previous steps. This strategy fits because exam analysis requires reasoning across multiple documents and years, not just a single classification task.

© UMHackathon 2026

4.3.3. **Context & Input Handling**: PDFs are extracted using pdf.js to plain text. For oversized documents exceeding GLM context limits, the system truncates input to the first 8,000 characters for lectures and 4,000 characters per past paper. Chapter detection and question matching operate on the truncated content, which typically captures the most important material from the beginning of the document. Lecture slides and past papers up to 50 pages are accepted. Large PDFs are processed using the first portion of extracted text. Very large documents (50+ pages) may have incomplete chapter detection for later sections.

4.3.4. **Fallback & Failure Behavior:** When GLM returns off-topic, unusable, or hallucinated responses, OracleSort implements graceful degradation. Each analysis step (matching, predictions) has independent error handling — if one step fails, partial results from successful steps are still saved. Failed files show a 'Failed' status badge. Users can delete failed entries and re-upload. The reference panel displays all matched questions so users can visually verify accuracy.

5. User Stories & Use Cases

- **User Stories:** As a SPM student in Kota Kinabalu, Sabah, my school is 2 hours from the city. We share one set of past year papers — 10 copies for 80 students. My teacher photocopies them by hand. She stays back until 7pm just to make sure we have something to study. I want to respect her effort. I want to study properly. But when I look at 10 years of Mathematics papers scattered across 8 chapters, I don't even know where to start. I tried sorting them once. I sat on the floor with scissors, glue, and printed papers. 6 hours later, I had glue on my uniform, papers everywhere, and I still mixed up Chapter 4 with Chapter 5. My mother asked me why I was crying. I told her I'm not sad. I'm just tired. I know the method works. But I cannot do this for 8 subjects. I cannot ask my teacher to do this for 80 students. I am one person.

- **Use Cases (Main Interactions):**

- **Upload Lecture Slides** — User uploads lecture PDFs → GLM detects chapters → System stores chapter data for future matching
- **Batch Upload Past Papers** — User uploads multiple past papers at once → Clicks "Analyze All Papers" → GLM matches every question to chapters → Dashboard displays trends and predictions
- **Export Separate PDFs** — User clicks Export button → Selects 'Separate PDFs by Chapter' → System generates individual PDF files for each chapter that has matched questions
- **Export Combined PDF** — User clicks Export button → Selects 'Combined PDF' → System generates a single merged PDF containing all chapters and predictions
- **View Reference** — User clicks on any prediction or question count → Slide-in reference panel appears → Shows source document name, original question text, and year of paper
- **AI Chat Assistant** — User clicks floating chat button → Types a question (e.g., "Why is Chapter 3 high priority?") → GLM responds with context-aware answer based on analysis data

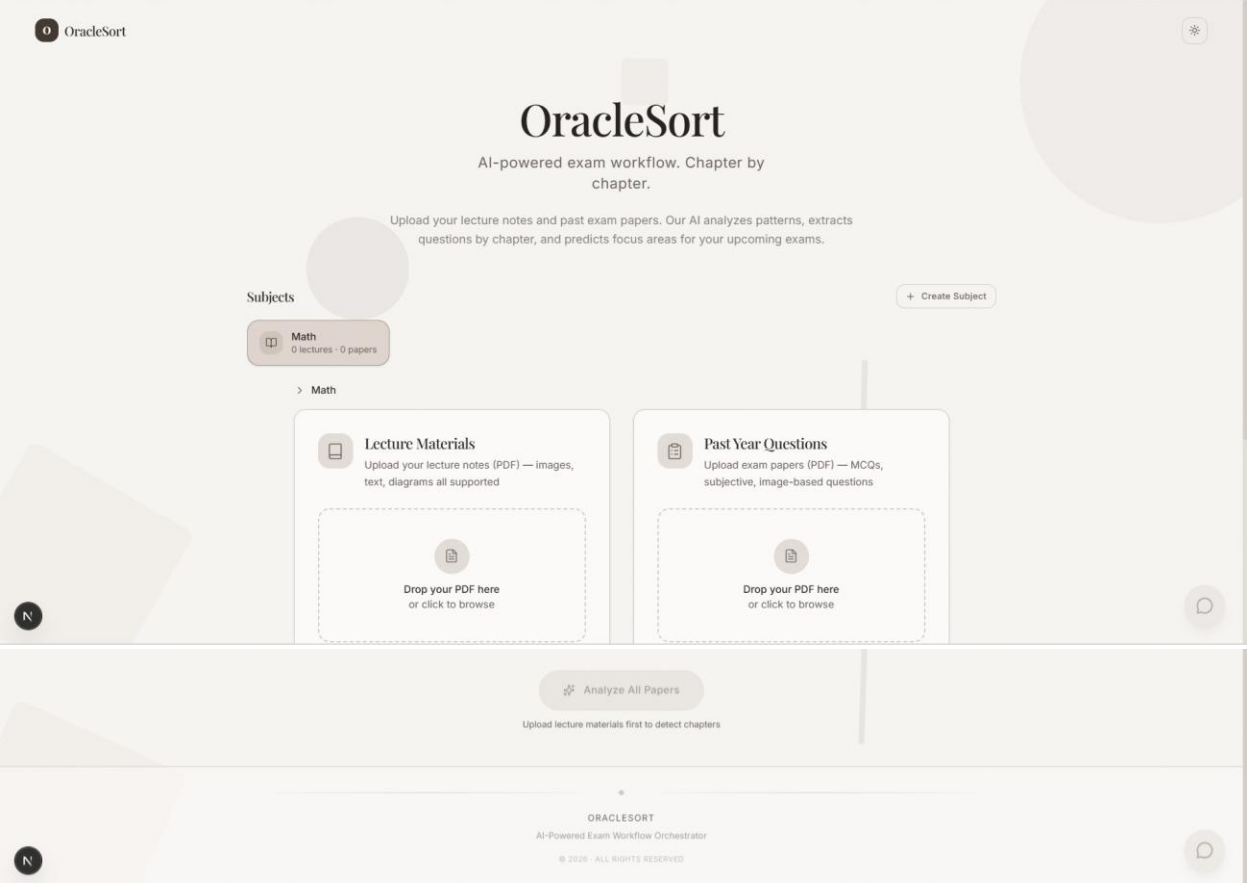
© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Use Case Diagram:** *(Optional but encouraged for participants to visualize the interaction)*.

```
                ┌──────────────────────────────────────────────────┐
                │        OracleSort System          │
                │                                   │
┌──────────────────────┐  │ • Create Subject                  │
│                      │  │ • Upload Lecture PDFs             │
│      Student         │──→│ • Upload Past Papers              │
│                      │  │ • Analyze All Papers              │
│                      │  │ • View Dashboard                  │
│                      │  │ • Export PDFs                     │
│                      │  │ • Ask AI Chatbot                  │
└──────────────────────┘  │                                   │
                │       ↑                           │
                │       │ <<includes>>              │
                │       │                           │
                │  ┌────────┴────────┐                  │
                │  │ GLM API         │                  │
                │  │ (Z.ai)          │                  │
                │  └─────────────────┘                  │
                └──────────────────────────────────────────────────┘
```

© UMHackathon 2026

Email: umhackathon@um.edu.my



6. Features Included (Scope Definition)

- **Subject Folders** — Create and manage subjects. Data saves automatically.
- **Batch Upload** — Upload multiple past year papers at once. No one-by-one.
- **One-Click Analysis** — Press one button. AI processes everything.
- **Chapter Detection** — AI reads lecture slides and finds all chapters**.**
- **Question Matching —** AI matches every past year question to the correct chapter.
- **Trend Dashboard** — See which chapters have the most questions. Instantly**.**
- **Prediction Cards —** High, Medium, Low priority. With AI reasoning.
- **Export by Chapter —** Download PDFs sorted by chapter. One click.
- **AI Chat Assistant —** Ask anything about your analysis. Get instant answers.

7. Features Not Included (Scope Control)

- **User Login —** No accounts. Data stays in browser only.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Cloud Storage** — No syncing across devices.
- **Answer Keys** — No answers generated. Only chapter matching.
- **Image Recognition —** Text-based PDF only.
- **Offline Mode —** Internet required for AI features.

8. Assumptions & Constraints

- **Internet Required** — AI analysis and chat need active internet. No offline mode.
- **Clean PDFs Only** — Scanned or low-quality PDFs may fail text extraction. Users need readable files.
- **Lecture Slides First** — Users must upload lecture slides before analyzing past papers for chapter detection.
- **Processing Takes Time** — Analysis takes 30-60 seconds for 10 papers. Loading animation keeps users informed.
- **Browser Storage Only** — Data lives in browser. Clearing cache deletes everything. Users should export important PDFs.
- **Modern Browser Needed** — Chrome, Firefox, Safari, Edge supported. Internet Explorer not supported.

9. Risks & Questions Throughout Development

- **Poor Quality PDFs** — If text extraction fails, system shows error message and skips that file. Other papers still process.
- **Wrong Chapter Matching** — Users can verify classifications in the reference panel. If matching is wrong, users can delete the subject and re-upload with clearer PDFs.
- **API Goes Down** — Last analysis results cached in browser. Demo still works with cached data. Failed files show 'Failed' status badge..
- **Storage Full** — If localStorage is full, saves may fail silently. Users should export important PDFs and delete old subjects regularly

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Slow Processing (30-60 sec)** — Four-step loading animation (Extracting → Matching → Analyzing → Generating) keeps users informed.
- **Internet Fails During Demo** — Pre-analyzed subject loaded with cached data. Dashboard, exports, reference panel work. Chat assistant will fail to respond if internet is down. Dashboard, exports, and reference panel still work with cached data.

© UMHackathon 2026